

DOMANDE TEORIA

a) (4) Illustrare le proprietà delle transazioni; si indichi quali moduli di un DBMS garantiscono ciascuna di tali proprietà.

Le proprietà delle transazioni sono quelle ACID, cioè:

- Atomicità: Ogni transazione deve essere eseguita per intero oppure non essere eseguita per niente
- Consistenza: Tutti i vincoli della base di dati devono essere rispettati dopo l'esecuzione della transazione
- Isolamento: Ogni transazione non influisce sull'esecuzione delle altre
- Persistenza: Il commit della transazione deve garantire la modifica della base di dati

I moduli che garantiscono ciascuna proprietà sono:

- Gestore dell'esecuzione concorrente: garantisce atomicità e isolamento
- Gestore dei metodi di accesso: garantisce la consistenza
- Gestore dell'affidabilità: garantisce l'atomicità e la persistenza

b) (4) Illustrare la struttura ad accesso calcolato (hash), in particolare specificar il funzionamento dell'operazione di inserimento e di ricerca di una tupla di chiave K ed evidenziare i casi in cui è più vantaggiosa di un B+-tree.

La struttura ad accesso calcolato posiziona i dati in memoria in base alla funzione di hashing che converte le chiavi in un indirizzo di memoria secondaria. Sono definite le seguenti funzioni:

- Funzione di folding: $f: K \rightarrow \mathbb{Z}^+$
Trasforma una chiave in un intero positivo
- Funzione di hashing: $h: \mathbb{Z}^+ \rightarrow B$
Trasforma una chiave foldata in un indirizzo di memoria di un bucket

I vantaggi di questa struttura sono che la ricerca ha costo costante, mentre lo svantaggio è che la funzione di hashing può avere collisioni, cioè per una stessa chiave si ottiene lo stesso valore di hash:

$$k_1 \in K, k_2 \in K, k_1 \neq k_2 \Rightarrow h(f(k_1)) = h(f(k_2))$$

Una buona funzione di hashing minimizza le collisioni.

- Per ricercare una tupla di chiave K si calcola il valore di hash della chiave e si segue il puntatore all'indirizzo di memoria, cioè un bucket, e si accede sequenzialmente alle tuple contenute.
- Per inserire una tupla di chiave K si calcola il valore di hash e si inserisce all'interno del bucket se è già esistente, altrimenti si crea un nuovo bucket. Se non c'è più spazio in un bucket già esistente si crea un bucket di overflow che però riduce le prestazioni.

La struttura ad accesso calcolato è più vantaggiosa rispetto ad un B+tree quando si hanno pochi inserimenti e tante ricerche perché ha costo costante per le ricerche. La struttura ad accesso calcolato fallisce nel caso in cui ci sono troppi dati all'interno dei bucket.

c) (5) Illustrare dettagliatamente il concetto di sharding in MongoDB: a cosa serve e perché si utilizza, come viene implementato e quali sono i componenti che lo gestiscono.

Lo sharding consiste nella separazione dei dati a livello di collezione in modo da avere una distribuzione dei dati gestita da più entità. Ogni shard è un replica set, cioè un insieme di nodi che replicano i dati in modo da garantire la affidabilità. In un replica set è presente un nodo primario che gestisce tutte le operazioni di scrittura salvandole in un oplog e più nodi secondari che replicano le operazioni del primario tramite lo scambio degli oplog. Gli shard sono gestiti da "mongos" che si occupa di delegare le query agli shard corretti in modo da ottimizzare i tempi e alla fine raggruppa tutti i risultati parziali per ritornare il risultato finale. Inoltre è presente un config server che mantiene i metadati riguardo allo sharding. Lo sharding serve ad aumentare la velocità di scrittura e lettura dei dati, aumentano la capacità di archiviazione. Inoltre permettono di sfruttare la località dei dati (zone sharding). Lo sharding viene implementato a livello di collezione e suddivide i documenti in chunk in base ad una shard key ottenuta da un insieme di dati dei documenti. Esistono due principali implementazioni:

- Ranged sharding: Suddivide i documenti in base ad un intervallo basato sul valore della chiave di sharding
- Hashed sharding: Viene calcolato un valore di hash in base allo shard key e i chunk sono suddivisi in base ad un intervallo su quel valore di hash

ESERCIZI

d) (4) Gestore dell'affidabilità

Si supponga che si verifichi un guasto di sistema con perdita di contenuto della memoria centrale. Al momento del guasto il contenuto del file di LOG è il seguente:

B(T1), B(T2), U(T1,O1,B1,A1), I(T2,O2,A2), B(T3), B(T4), U(T4,O2,B3,A3), U(T3,O1,B4,A4), A(T1), CK(T2,T3,T4), C(T4), B(T5), U(T3,O3,B5,A5), U(T5,O4,B6,A6), D(T2,O5,B7), C(T2), A(T5), I(T3,O6,A8) guasto

Illustrare cosa accade al riavvio del sistema nell'esecuzione della procedura di ripresa a caldo.

- 1 $Undo = \{T_2, T_3, T_4\}$ $Redo = \{\}$
- 2 $Undo = \{\cancel{T_2}, \cancel{T_3}, \cancel{T_4}, T_5\}$ $Redo = \{T_4, T_2\}$
- 3 $Undo = \{T_3, T_5\}$ $Redo = \{T_4, T_2\}$

Undo

$$I(T_3, O_6, A_8) \Rightarrow Delete(O_6)$$
$$U(T_5, O_4, B_6, A_6) \Rightarrow O_4 := B_6$$
$$U(T_3, O_3, B_5, A_5) \Rightarrow O_3 := B_5$$
$$U(T_3, O_1, B_4, A_4) \Rightarrow O_1 := B_4$$

4 Redo

$$I(T_2, O_2, A_2) \Rightarrow O_2 := A_2$$
$$U(T_4, O_2, B_3, A_3) \Rightarrow O_2 := A_3$$
$$D(T_2, O_5, B_7) \Rightarrow Delete(O_5)$$

f) Ottimizzazione

Si consideri il seguente schema relazionale contenente i risultati di elezioni :

STUDENTE(Matricola, Nome, Cognome, Indirizzo, Comune, Provincia, Regione, Email);

ESAME(Studente, Docente, Data, Orainizio, Durata, Voto)

DOCENTE(CodiceFiscale, Cognome, Nome, Qualifica)

Vincoli d'integrità referenziale: ESAME.Studente -> STUDENTE, ESAME.Docente -> DOCENTE

Data la seguente interrogazione

```
SELECT S.Matricola, S.Cognome, S.Nome, E.Docente, E.Data, E.Orainizio, E.Voto
FROM STUDENTE S JOIN ESAME E ON (S.Matricola = E.Studente)
WHERE S.Provincia = 'Verona' AND E.Data > '01/01/2023'
```

(3) Calcolare il costo dell'interrogazione in termini di numero di accessi a memoria secondaria sotto le seguenti ipotesi:

- la selezione degli studenti viene eseguita attraverso una scansione della tabella STUDENTE, il risultato della selezione viene mantenuto nel buffer
- la selezione degli esami viene eseguita con una scansione sequenziale della tabella ESAME, il risultato viene salvato in 1750 pagine della memoria secondaria (201500 righe).
- l'ordine di esecuzione del join è STUDENTE $\bowtie$ ESAME e le operazioni di join vengono eseguite con la tecnica "Nested Loop Join" con una pagina di buffer disponibile per ogni tabella
- NP(DOCENTE) = 14, NP(ESAME) = 2500, NP(STUDENTE) = 250
NR(DOCENTE) = 1700, NR(ESAME) = 294500, NR(STUDENTE) = 15500
- VAL(Provincia, STUDENTE) = 10, VAL(Studente, ESAME) = 15500

Le operazioni da eseguire sono:

1. Selezione su Studente, siccome bisogna caricare tutta la tabella studente il costo è:

$$NP(Studente) = 250$$

il risultato viene mantenuto nel buffer, quindi il costo di scrittura è 0.

2. Selezione su Esame, siccome bisogna caricare tutta la tabella studente, il costo è:

$$NP(Esame) = 2500$$

il risultato viene salvato in 1750 pagine di memoria secondaria (201500 righe)

3. Studente join Esame, bisogna caricare la selezione su studente, che costa 0 perchè è già nel buffer, e bisogna anche caricare la selezione su Esame, quindi il costo è:

$$NP(Sel.Verona(Studente)) + NR(Sel.Verona(Studente)) \cdot NP(Sel.Data(Esame))$$

Bisogna calcolare il numero di righe della selezione su studente che è dato dalla selettività:

$$NR(Sel.Verona(Studente)) = \frac{NR(Studente)}{Val(Provincia, studente)} = \frac{15500}{10} = 1550$$

Il costo finale del join è:

$$NP(Sel.Verona(Studente)) + NR(Sel.Verona(Studente)) \cdot NP(Sel.Data(Esame))$$

$$= 0 + 1550 \cdot 1750 = 2625000$$

Il costo totale della query è la somma delle singole operazioni:

$$250 + 2500 + 2625000 = 2627750$$

(2) Come cambia il costo della query considerando la presenza di un indice B+-tree di profondità 3 sull'attributo Studente di ESAME.

L'aggiunta dell'indice cambia il costo soltanto della join, quindi il costo delle altre operazioni rimane invariato.

3. Studente join Esame, bisogna caricare la selezione su studente, che costa 0 perchè è già nel buffer, e bisogna anche caricare la selezione su Esame, quindi il costo è:

PI=3

$$NP(Sel.Verona(Studente)) + NR(Sel.Verona(Studente)) \cdot \left(PI + \frac{NR(Sel.Data(Esame))}{Val(Studente, Esame)} \right)$$

Bisogna calcolare la selettività di esame sull'indice:

$$\frac{NR(Sel.Data(Esame))}{Val(Studente, Esame)} = \frac{201500}{15500} = 13$$

Quindi il costo finale della join è:

$$0 + 1550 \cdot (3 + 13) = 1550 \cdot 16 = 24800$$

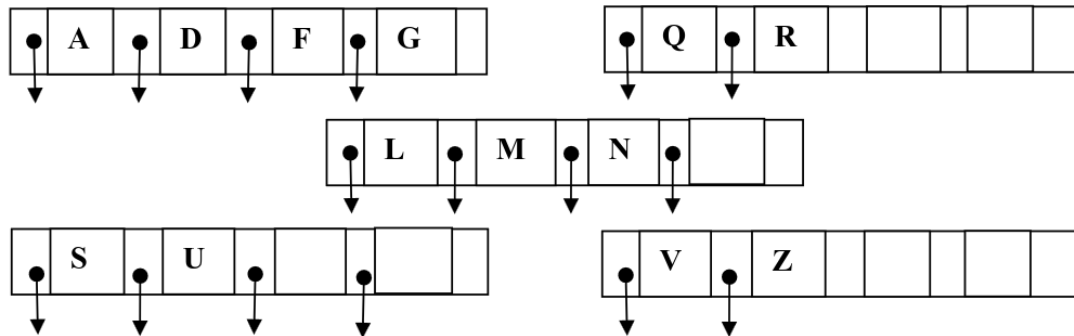
Il costo totale della query è:

$$250 + 2500 + 24800 = 27550$$

g) B+-tree

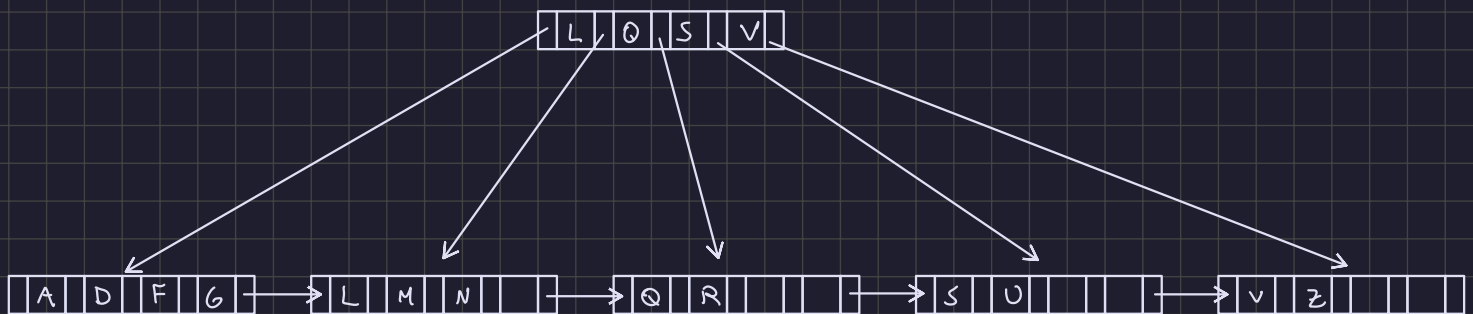
Data la seguente lista di possibili valori chiave $L = (A, B, C, D, E, F, G, H, I, L, M, N, O, P, Q, R, S, T, U, V, Z)$:

a) (1) costruire un B+-tree (fan-out=5) che contenga i seguenti nodi foglia:

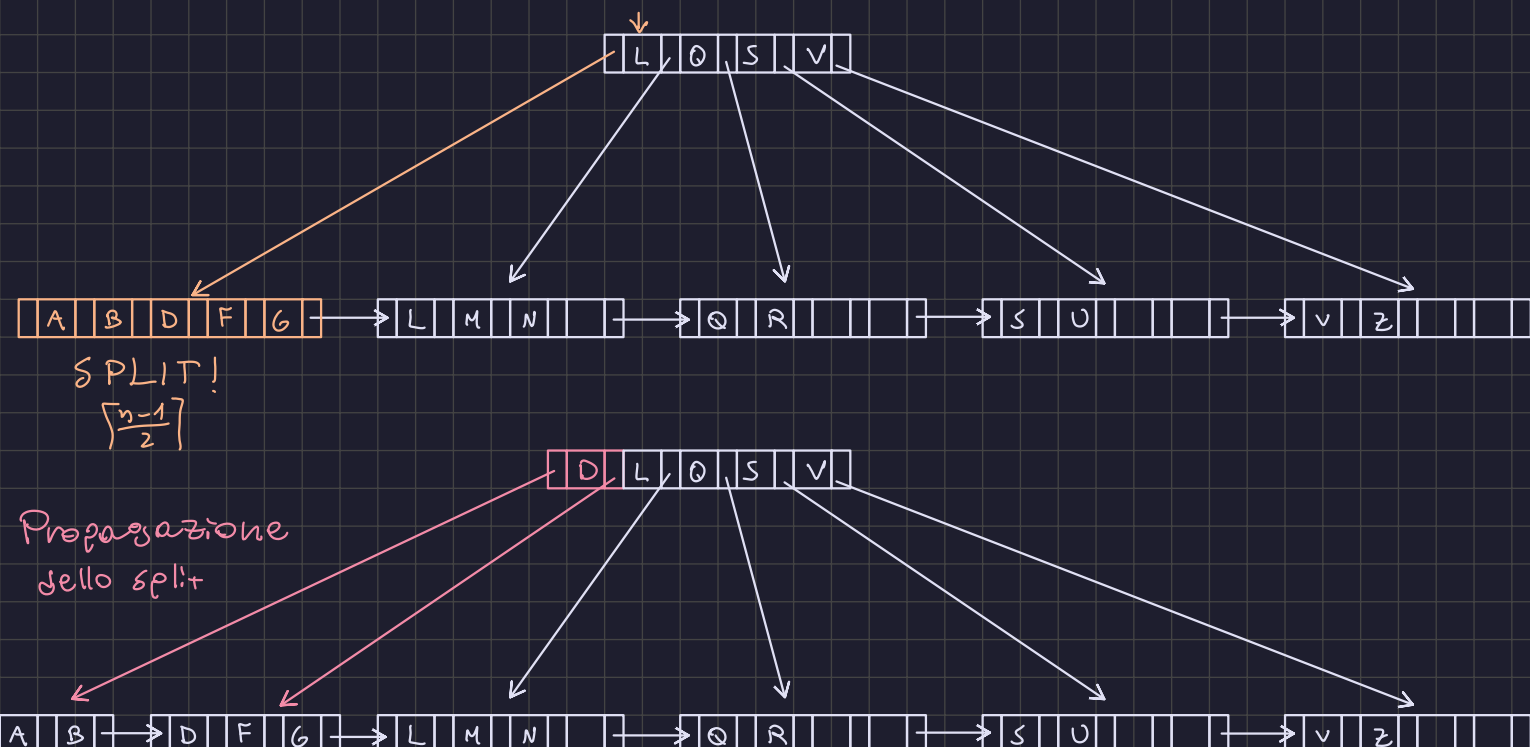


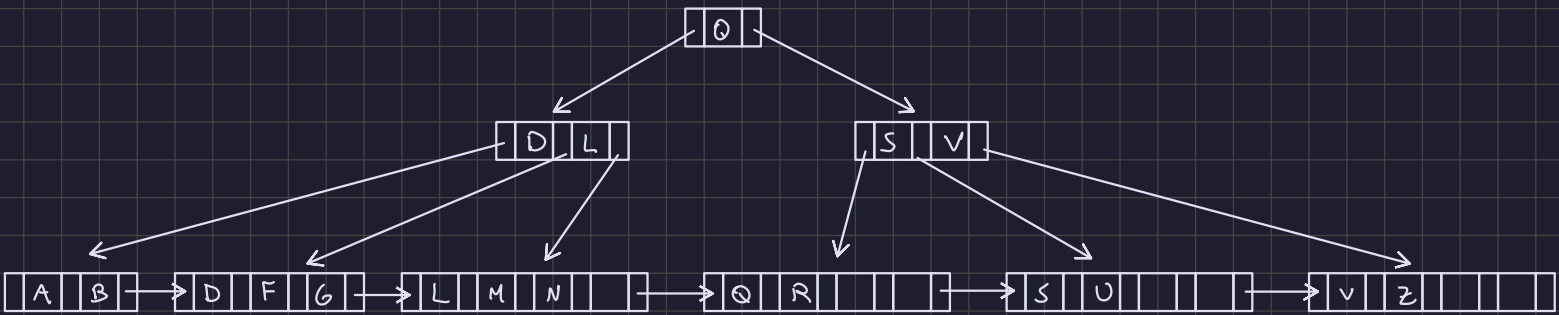
Vincoli di chiave: $2 \leq \# \text{chiavi} \leq 4$

Vincoli di puntatori: $3 \leq \# \text{puntatori} \leq 5$

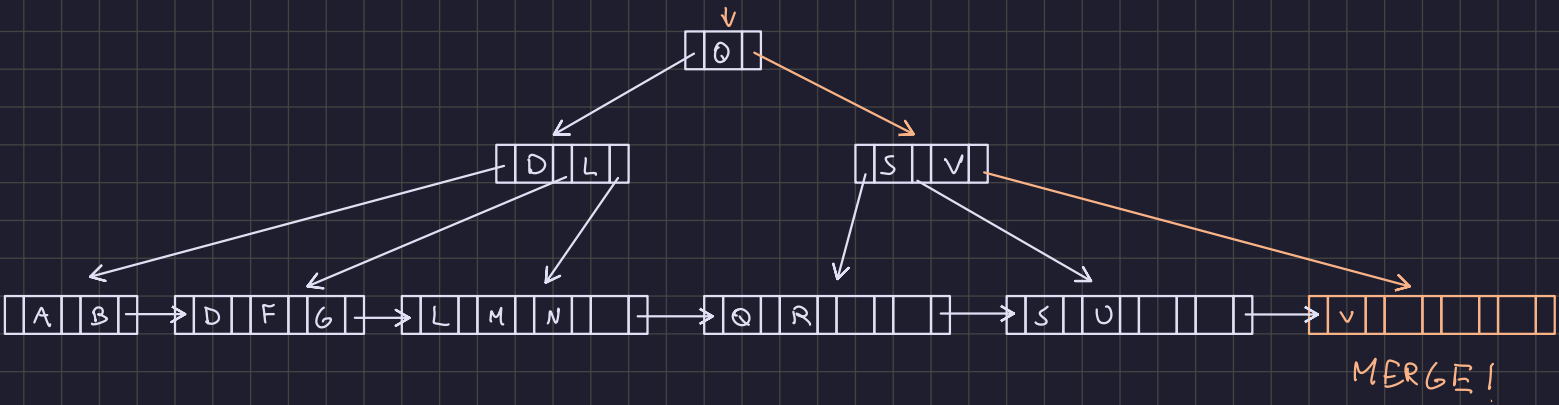


b) (2) mostrare l'albero dopo l'inserimento del valore chiave B



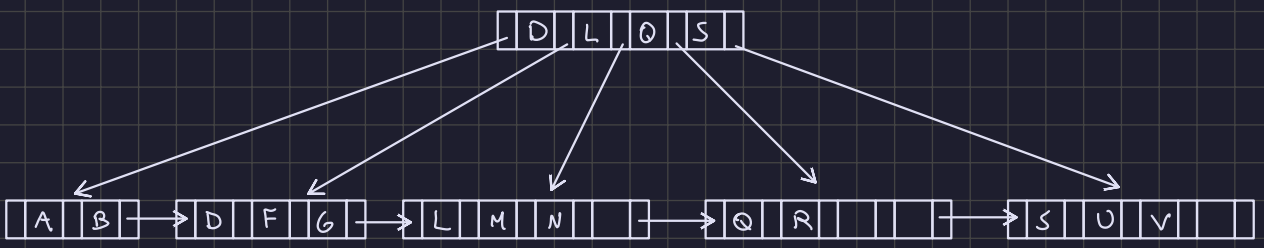
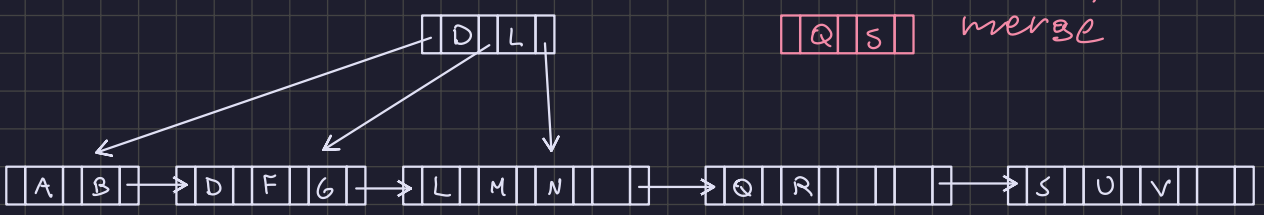


c) (2) partendo dal risultato del punto precedente mostrare l'albero dopo la rimozione del valore chiave Z.



MERGE!

Propagazione del
[Q, S] merge



e) Esecuzione concorrente

Dato il seguente schedule S:

S: r2(y), w3(z), r2(x), w1(z), w2(y), w3(t), r3(y), w3(x), r1(x), r1(y), w0(x), w1(t)

- (2) Indicare se S è view-serializzabile (VSR) oppure no (giustificare la risposta)
- (2) Indicare se S è conflict-serializzabile (CSR) oppure no (giustificare la risposta)
- (2) Indicare se S è 2PL (vale a dire, rispetta la regola di serializzabilità del locking a due fasi)

- S è 2PL

S: $r_2(y), w_3(z), r_2(x), w_1(z), w_2(y), w_3(t), r_3(y), w_3(x), r_1(x), r_1(y), w_0(x), w_1(t)$

	$r_2(y)$	t2 fa r_lock(y)
	$w_3(z)$	t3 fa w_lock(z)
x	$r_2(x)$	t2 fa r_lock(x)
	$w_1(z)$	t1 fa w_lock(z)
	$w_2(y)$	t2 rilascia il r_lock su y e fa w_lock(y)
x	$w_3(t)$	t3 fa w_lock(t)
	$r_3(y)$	t3 aspetta che t2 rilasci il w_lock(y) e quando lo fa t3 esegue r_lock(y). t2 può rilasciare il lock perchè non ha più operazioni.
x	$w_3(x)$	t3 aspetta che t2 rilasci il r_lock(x) e quando lo fa t3 esegue w_lock(x). t2 può rilasciare il lock perchè non ha più operazioni.
x	$r_1(x)$	t1 aspetta che t3 rilasci il w_lock(x) e quando lo fa t1 esegue r_lock(x). t3 può rilasciare il lock perchè non ha più operazioni.
	$r_1(y)$	t1 fa r_lock(y), può farlo perchè read lock può essere fatto più volte sulla stessa risorsa
x	$w_0(x)$	t0 aspetta che t1 rilasci il r_lock(x) e quando lo fa t0 esegue r_lock(x). t1 può rilasciare il lock perchè non ha più operazioni.
x	$w_1(t)$	t1 aspetta che t3 rilasci il w_lock(t) e quando lo fa t1 esegue w_lock(t). t3 può rilasciare il lock perchè non ha più operazioni.

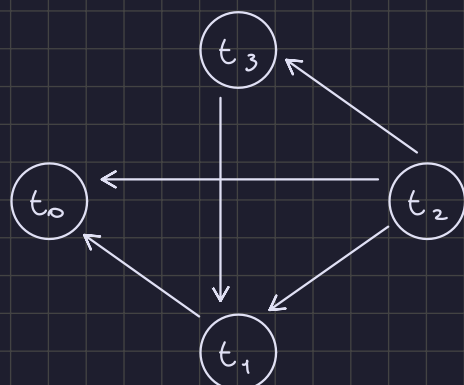
Lo schedule S rispetta i vincoli per il 2PL, quindi è 2PL e di conseguenza anche view-serializzabile e conflict-serializzabile perchè:

$$VSR \subset CSR \subset 2PL$$

- Conflict serializzabilità (Ridondante, perchè già controllato nel 2PL):
Inizializziamo l'insieme dei conflitti. Due transazioni sono in conflitto se:
 - Sono diverse
 - Almeno una delle due scrive
 - Operano sulla stessa risorsa
 - Quella a sinistra compare prima di quella a destra

S: $r_2(y), w_3(z), r_2(x), w_1(z), w_2(y), w_3(t), r_3(y), w_3(x), r_1(x), r_1(y), w_0(x), w_1(t)$

$$Conflietti = \{(r_2(x), w_3(x)), (r_2(x), w_0(x)), (w_3(x), r_1(x)), (r_1(x), w_0(x)), (w_2(y), r_3(y)), (w_2(y), r_1(y))\}$$



Siccome il grafo è aciclico, lo schedule S è conflict-serializzabile, e quindi è anche view-serializzabile perchè $CSR \subset VSR$

- View serializzabilità (Ridondante, perchè già controllato nella conflict-serializzabilità e 2PL):
Inizializziamo gli insiemi:
 - Legge_da: Una transazione legge da un'altra se una scrittura è preceduta da una lettura senza altre scritture in mezzo sulla stessa risorsa
 - Scritture_finali: Una scrittura è finale se non ci sono più altre modifiche della stessa risorsa successivamente

$$\text{Legge_da} = \{ (r_3(y), w_2(y)), (r_1(x), w_3(x)), (r_1(y), w_2(y)) \}$$

$$\text{Scritture_finali} = \{ w_1(t), w_0(x), w_2(y), w_1(z) \}$$

Due schedule sono view serializzabili se esiste una schedule seriale che ha gli stessi insiemi Legge_da e Scritture_finali. Verifichiamo i vincoli imposti sulle transazioni:

- t3 deve essere dopo t2 perchè r3(y) legge da w2(y)
- t1 deve essere dopo t3 perchè r1(x) legge da w3(x)
- t1 deve essere dopo t2 perchè r1(y) legge da w2(y)
- t1 deve essere dopo t3 perchè w1(t) è finale rispetto a w3(t)
- t0 deve essere dopo t3 perchè w0(x) è finale rispetto a w3(x)
- t1 deve essere dopo t3 perchè w1(z) è finale rispetto a w3(z)

Quindi una schedule seriale dovrebbe essere il seguente:

$$S' = t_2 \ t_3 \ t_1 \ t_0$$

$$= r_2(y), r_2(x), w_2(y), w_3(z), w_3(t), r_3(y), w_3(x), \\ w_1(z), r_1(x), r_1(y), w_1(t), w_0(x)$$

Inizializziamo gli insiemi Legge_da e Scritture finali per S':

$$\text{Legge_da} = \{ (r_3(y), w_2(y)), (r_1(x), w_3(x)), (r_1(y), w_2(y)) \}$$

$$\text{Scritture_finali} = \{ w_0(x), w_2(y), w_1(z), w_1(t) \}$$

Siccome gli insiemi di S' sono uguali a S, allora S è view-serializzabile.